Priority Queues

A priority queue is a special type of queue that allows us to keep a prioritized list of items.

In a priority queue, each item you insert into the queue has a "priority rating" indicating how important it is.

Any time you dequeue an item from a pri[illegible] always dequeues the item with the hig[illegible] (instead of just the first item in[illegible]

Example: If I have a queue of patients in the emergency room, I don't just take the next patient in line, I take the one who has the most severe injuries.

Priority Queues

A Priority Queue supports three operations:

- Insert a new item into the queue
- Get the value of the highest priority item
- Remove the highest priority item from the queue

When you define a Priority Queue, you must specify how to determine the priority of each item in the queue.

Priority = amount of blood lost + number of cuts

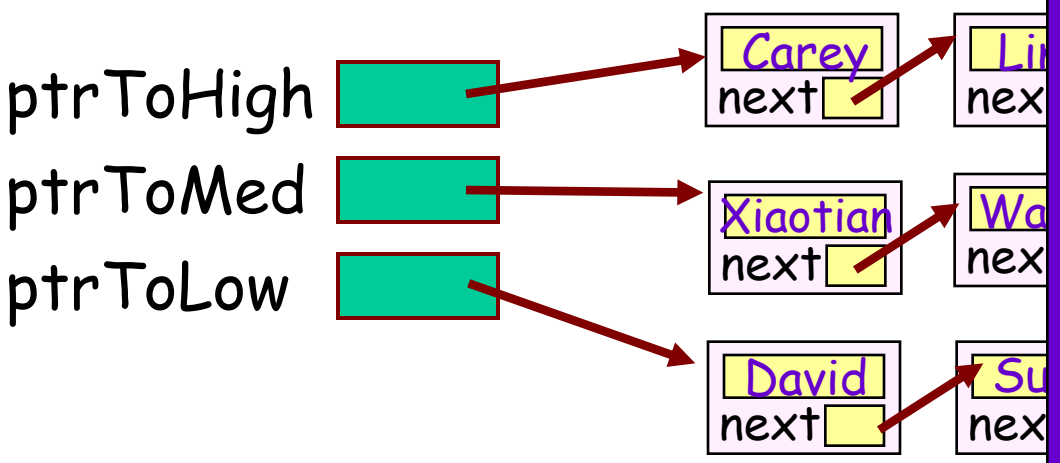
You must then design your PQ data structure/algorithms so you can efficiently retrieve the highest-priority item.

Priority Queues

Question: What data structures can we use to implement a priority queue? Hmmm…

Let's make it easier… What if we have just a limited set of priorities, e.g.: high, medium low?

Hint: Think of an airport ticket line with first, business and coach (cattle) class…



Right – we can use n linked lists, one for each priority level.

To obtain the highest-priority item, always take the first item from the highest priority, non-empty list.

Priority Queues

Question: Ok, but what data structure should we use if we have a huge number of priorities? Hmmm!

The HEAP data structure is one of the most efficient ones we can use to implement a Priority Queue.

The heap data structure uses a special type of binary tree to hold its data.

As we'll see, while a heap does use a binary tree to store its data, a heap is NOT a binary search tree.

The Heap ADT

What's the big picture?

A heap is a tree-based data structure used to implement priority queues and do efficient sorting*.

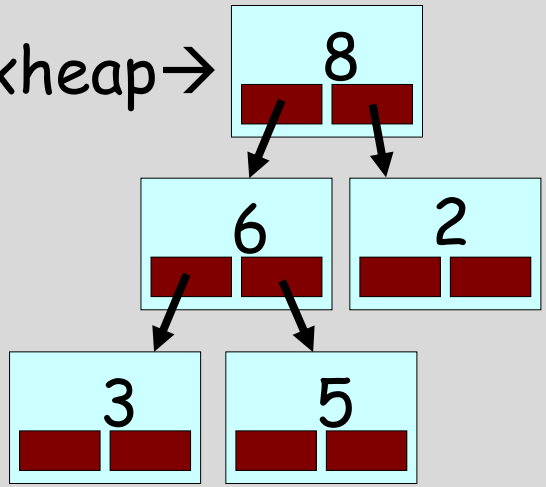
There are two types of heaps:
minheaps and maxheaps

In a minheap, the smallest item is always at the tree's root. Every time you extract or add an item, you update the tree to maintain this property.

(In a maxheap, the largest item is at the root.)

Due to clever use of an array to implement the tree, it's fast - $O(\log_2 n)$ - to keep the proper item at the top when you extract/insert items!

Maxheap→



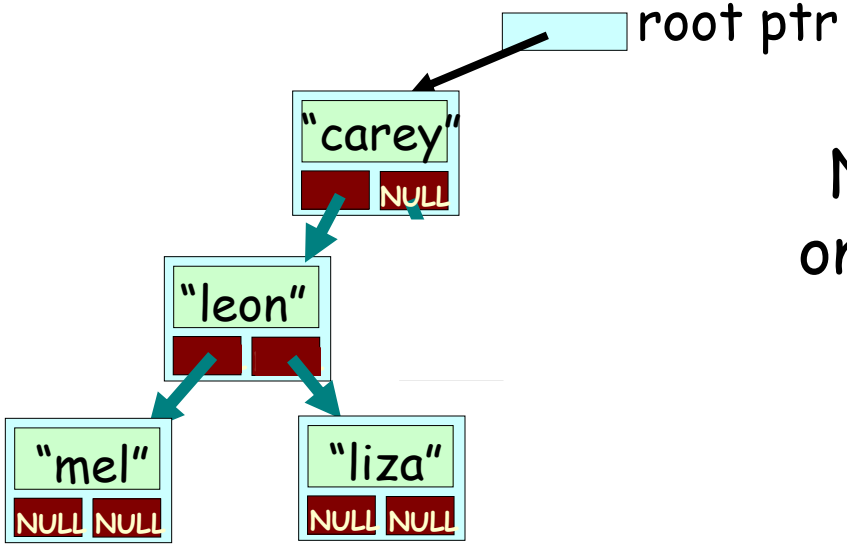
Uses:

Prioritize processing of data, find the shortest path for route navigation, sort arrays efficiently, merge streams of ordered data, etc.

All Heaps Use a "Complete" Binary Tree

A complete binary tree is one in which:

- The top N-1 levels of the tree are completely filled with nodes
- All nodes on the bottom-most level must be as far left as possible (with no empty slots between nodes!)



Is it complete?

No. The leaves on the bottom level are not as far left as possible!

Heaps… of…

A heap is a special type of complete binary tree
(it's **not** a binary search tree).

There are two types of heaps, minheaps and maxheaps:

Maxheap:

1. Quickly insert a new item into the heap
2. Quickly retrieve the largest item from the heap

Minheap:

1. Quickly insert a new item into the heap
2. Quickly retrieve the smallest item from the heap

The Maxheap

A maxheap is a binary tree which follows these rules:

1. The value contained by a node is ALWAYS GREATER THAN OR EQUAL TO the values of the node's children.

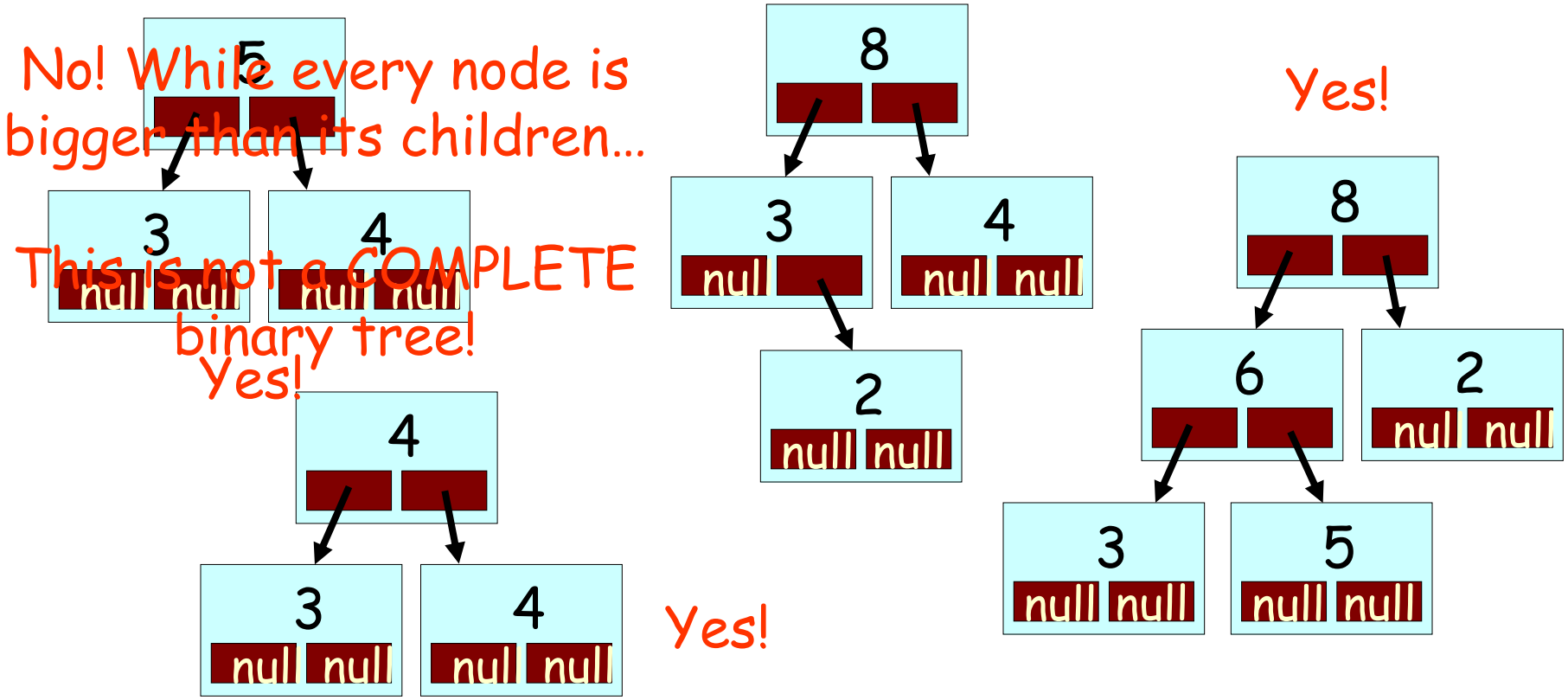
2. The tree is a COMPLETE binary tree.

Question: What are the rules for a minheap?

Complete Binary Tree:
Every level except the deepest level must contain as many nodes as possible, and the at the deepest level, all nodes are as far left as possible.

The Maxheap

Which of the following are valid maxheaps?



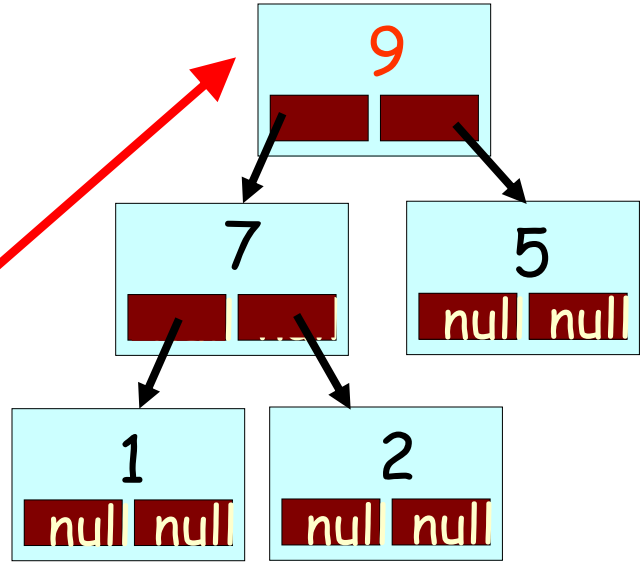
1. The value contained by a node is ALWAYS >= the values of the node's children.
2. The tree is a COMPLETE binary tree.

The Maxheap

One thing you'll notice about a maxheap is...

that, by definition, the biggest (highest priority) item

is always at the *root* (top) of the tree!



This means its easy to always find the biggest (highest priority) item in just a single step!

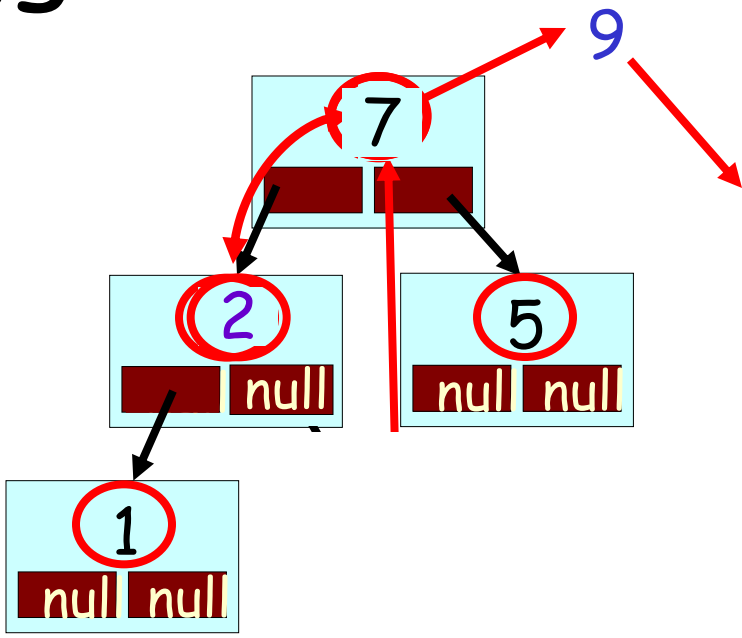
1. The value contained by a node is ALWAYS >= the values of the node's children.
2. The tree is a COMPLETE binary tree.

Operations on a Maxheap

Allright, now let's see how to extract an item from a heap and to add a new item to a heap!

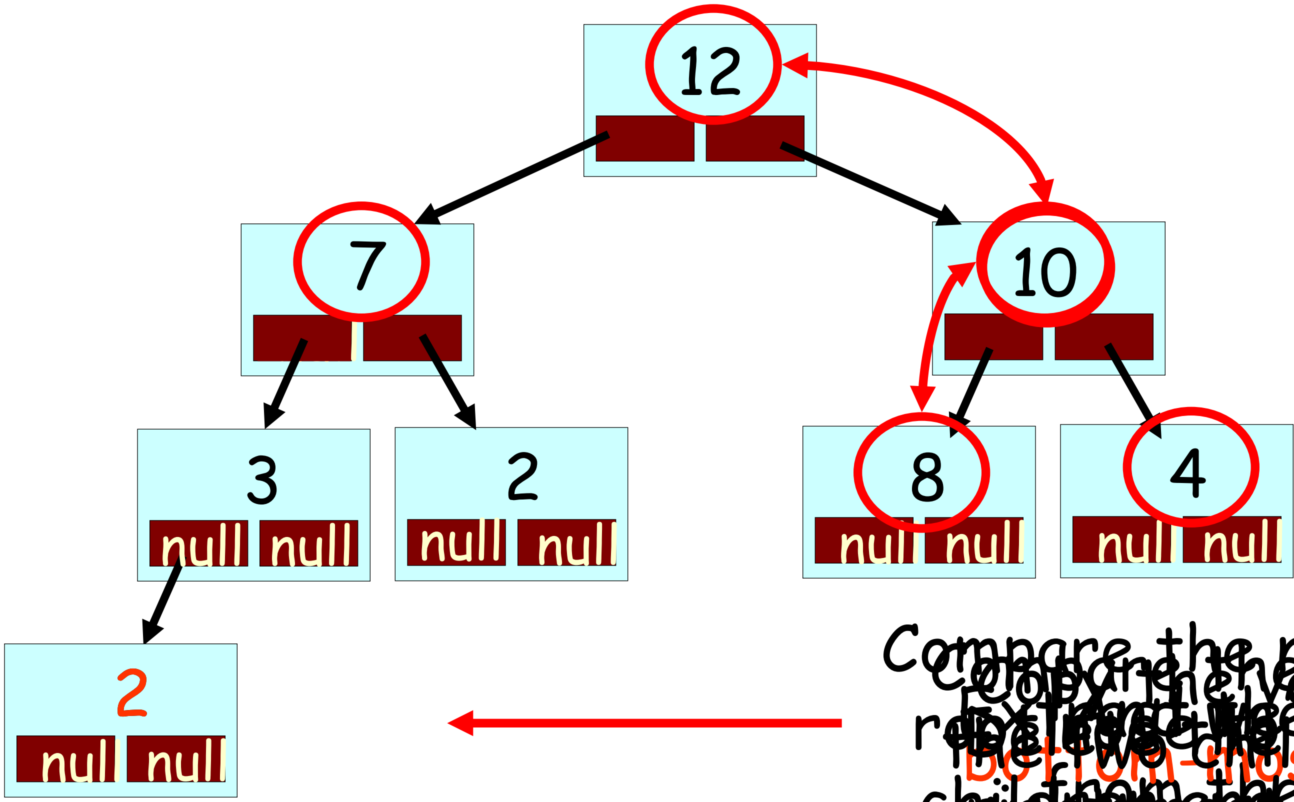
Extracting the Biggest Item

1. If the tree is empty, return error.
2. Otherwise, the top item in the tree is the biggest value. Remember it for later.
3. If the heap has only one node, then delete it and return the saved value.
4. Copy the value from the right-most node in the bottom-most row to the root node.
5. Delete the right-most node in the bottom-most row.
6. Repeatedly swap the just-moved value with the larger of its two children until the value is greater than or equal to both of its children. ("sifting DOWN")
7. Return the saved value to the user.



When we're done, the largest value is on the top again, and the heap is consistent.

Extraction Challenge!



Compare the new value in the

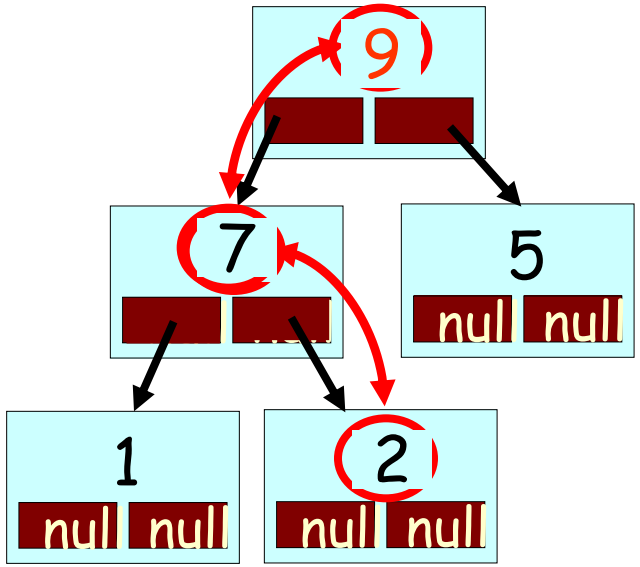
maxheap again!

Show the resulting heap after extracting the largest item!

Adding a Node to a Maxheap

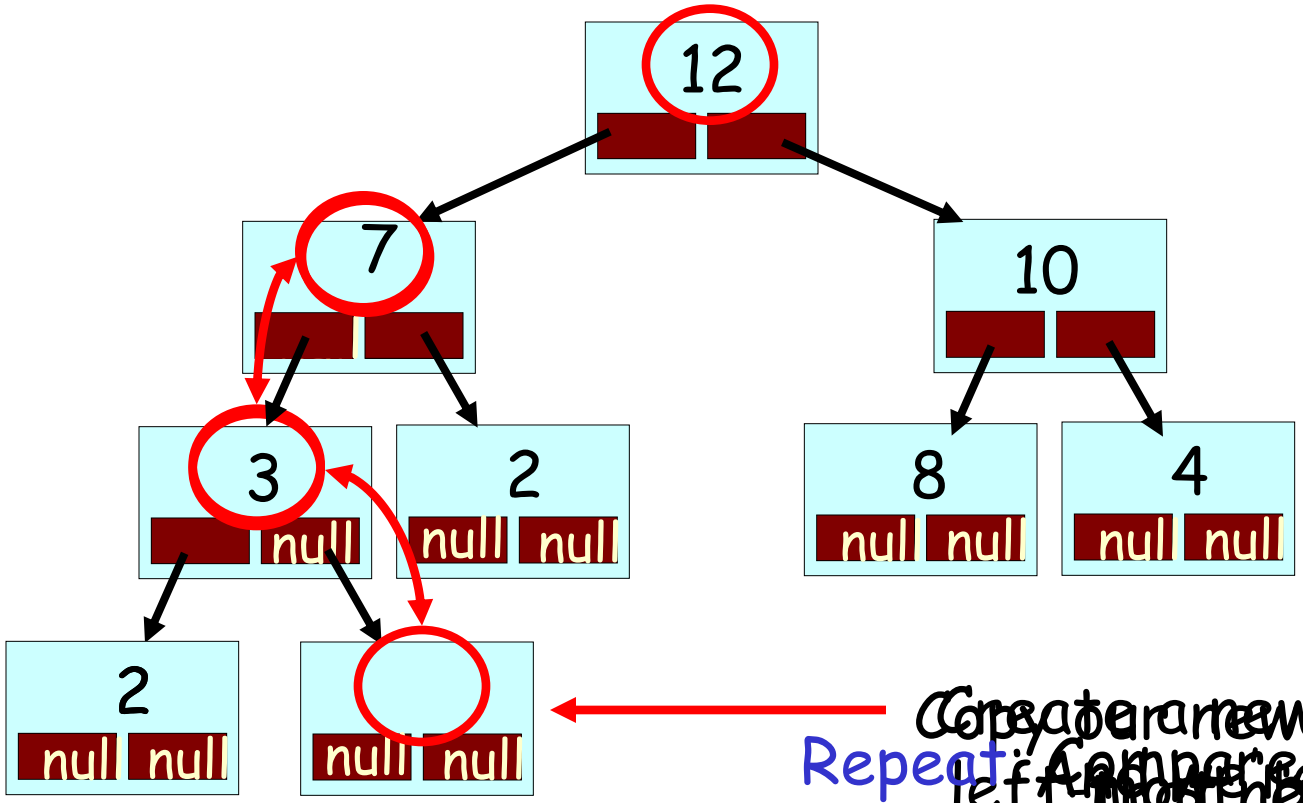
(Let's see how to add a value of 9)

1. If the tree is empty, create a new root node & return.
2. Otherwise, insert the new node in the bottom-most, left-most position of the tree (so it's still a complete tree).
3. Compare the new value with its parent's value.
4. If the new value is greater than its parent's value, then swap them.
5. Repeat steps 3-4 until the new value rises to its proper place.



This process is called "reheapification."

Insertion Challenge!



Create a new node in the left-most position possible (to ensure our tree stays *complete*)

Copy our new value, 9, into the new node.

Repeat: Compare our new value with the parent node's value and swap if our new value is bigger.

Show the resulting heap after inserting a value of **9**!

Implementing A Heap

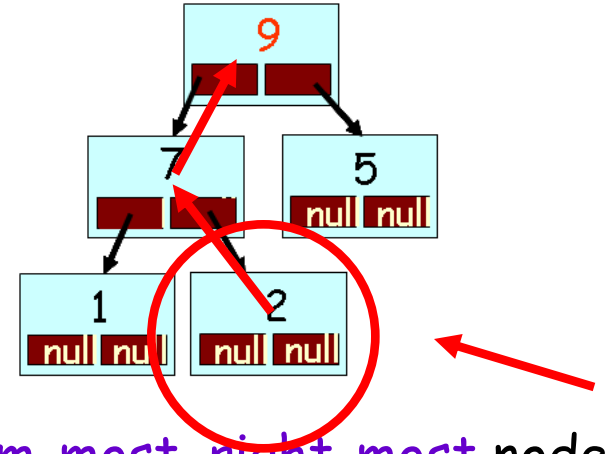
Question:

What data structure can we use to implement a heap?

How about a classical binary tree node with links?

```
struct node
{
   int value;
   node *left, *right;
};
```

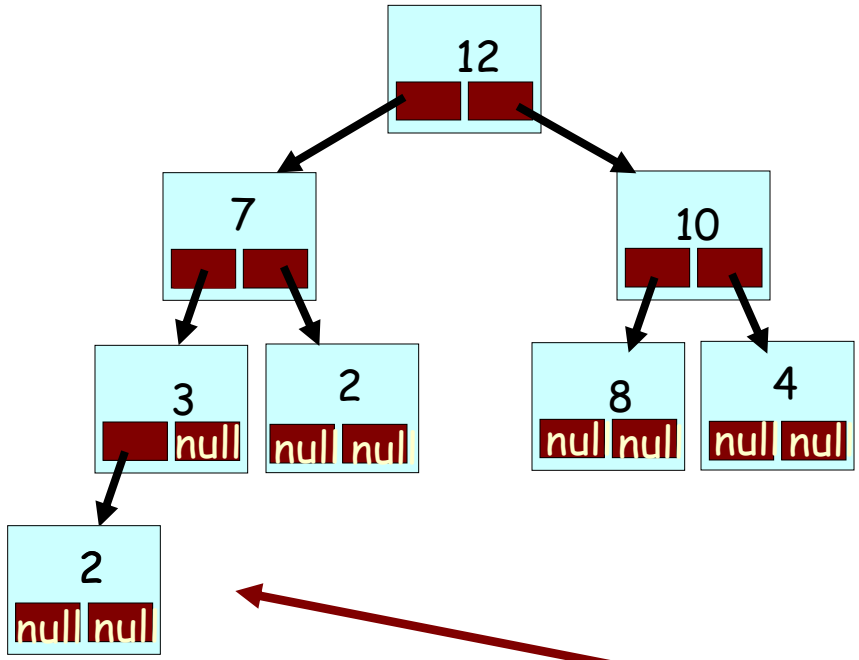
Hmmm… But this has some challenges. What are they?



1. It's not easy to locate the bottom-most, right-most node during extraction.
2. It's not easy to locate the bottom-most, left-most open spot to insert a new node during insertion!
3. It's not easy to locate a node's parent to do reheapification swaps.

Implementing A Heap

Perhaps there's some better data structure we could use…



Hmmmm. What about an array?

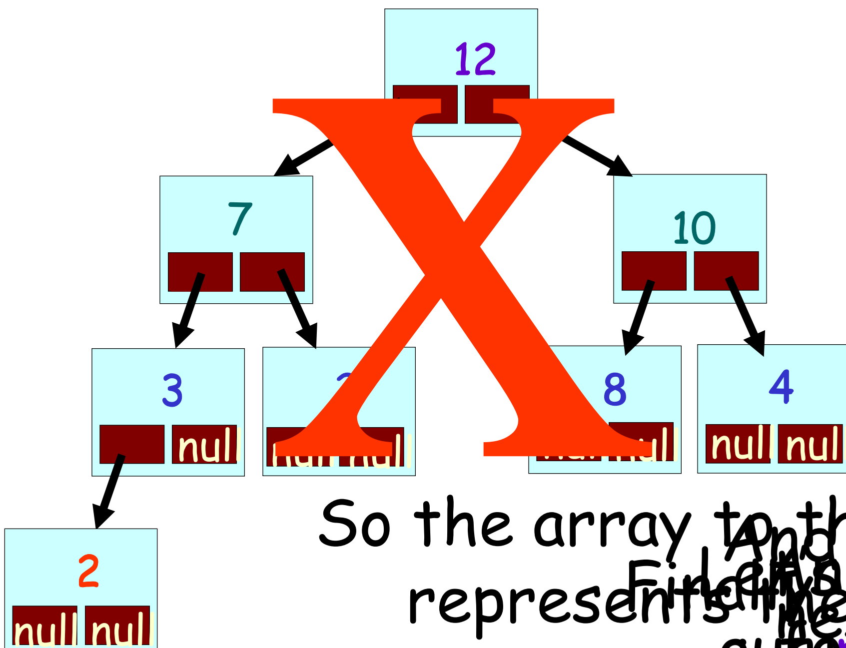
Well, we know that each level of our tree has 2x the number of nodes of the previous level*.

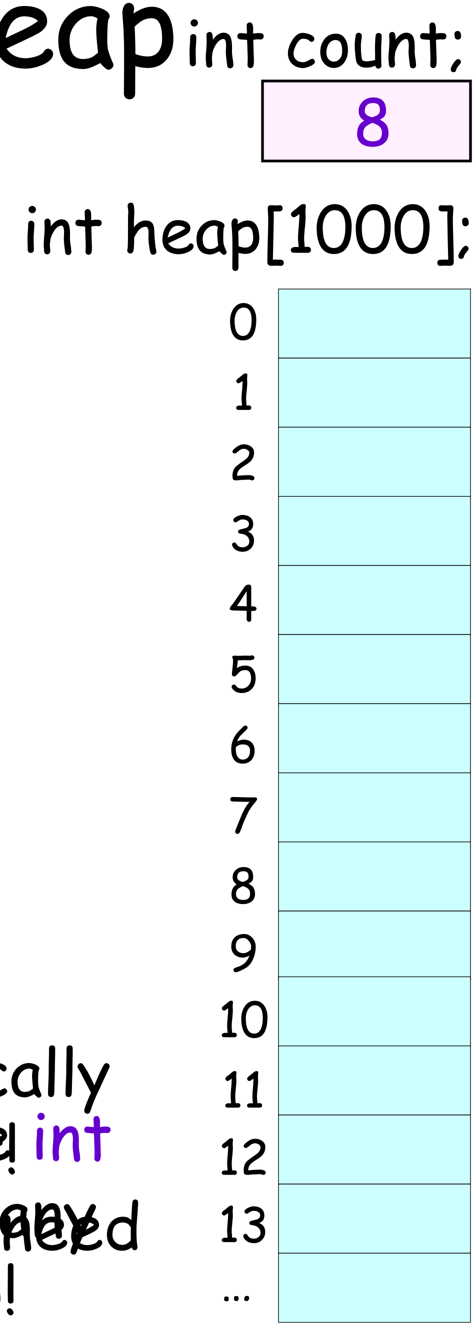
So what if we just copy our nodes a level at a time into an array???

* Except for the last level…

Implementing A Heap

So what if we just copy our nodes a level at a time into an array???





So the array to the right now logically represents [illegible] int
And if we use [illegible] need
to use [illegible] heap!

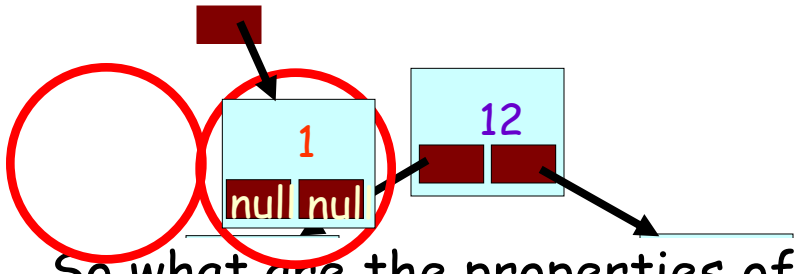
Implementing A Heap

int count;

9 8

int heap[1000];

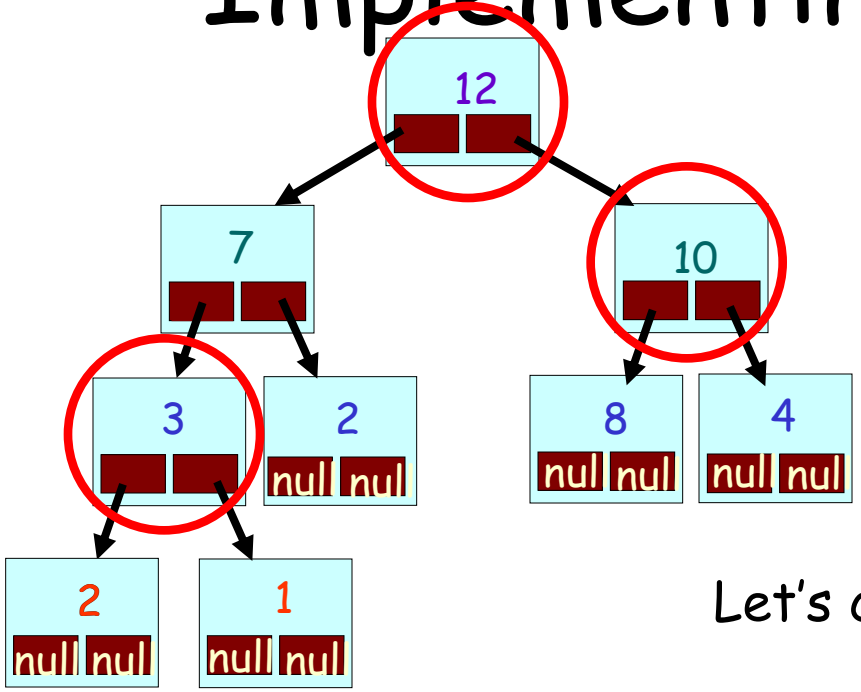
0	12
1	7
2	10
3	3
4	2
5	8
6	4
7	2
8	1
9	
10	
11	
12	
13	
…	



So what are the properties of our array-based tree?

1. We can always find the root value in heap[0]
2. We can always find bottom-most, right-most node in heap[count-1]
3. We can always find the bottom-most, left-most empty spot (to add a new value) in heap[count]
4. We can add or remove a node by simply setting heap[count] = value; and/or updating our count!

Implementing A Heap



int count;

9

int heap[1000];

0	12
1	7
2	10
3	3
4	2
5	8
6	4
7	2
8	1
9	
10	
11	
12	
13	

Ok, in our array, how do we locate the left and right children of a node?

Let's consider some examples:

Parent Slot#	Left Child Slot#	Right Child Slot#
0	1	2
2	5	6
3	7	8

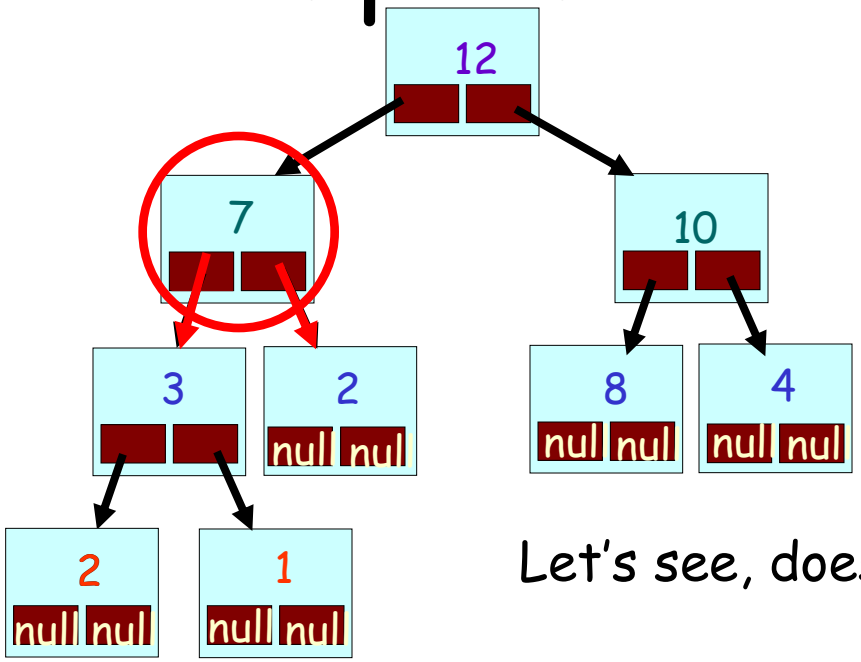
Challenge: Come up with a formula to locate a node's children

Hint: It's of the form

leftChild(parent) = 2 * parent + 1

rightChild(parent) = 2 * parent + 2

Implementing A Heap



int count;

9

int heap[1000];

0	12
1	7
2	10
3	3
4	2
5	8
6	4
7	2
8	1
9	
10	
11	
12	
13	
...	

Let's see, does our formula work?

Consider this node, which is in slot #1 of our array?

leftChild(1) = 2 * 1 + 1 = slot 3

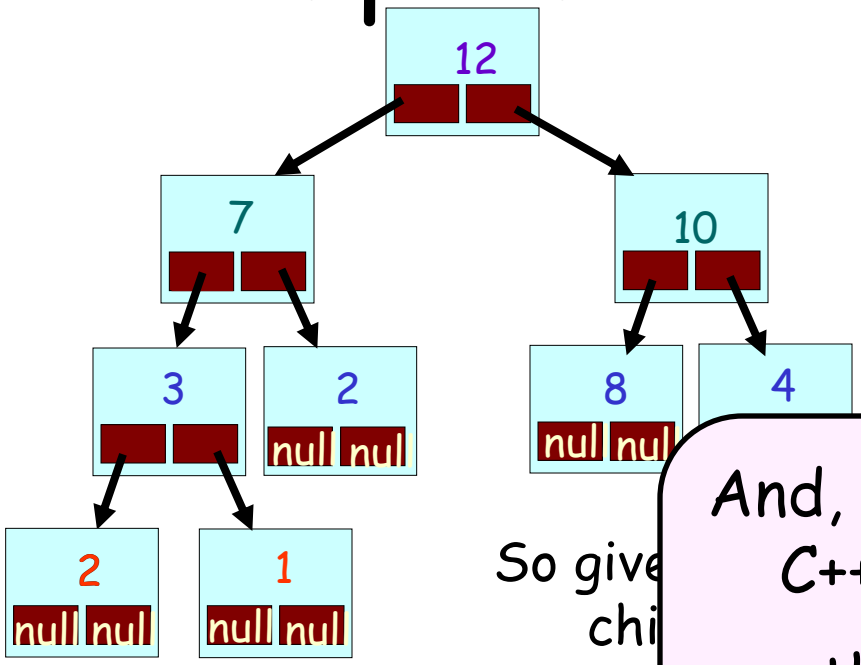
rightChild(1) = 2 * 1 + 2 = slot 4

Our formula appears to work!

Hint: It's of the form leftChild(parent) = 2 * parent + 1

rightChild(parent) = 2 * parent + 2

Implementing A Heap



int count;

9

int heap[1000];

0	12
1	7
2	10
3	3
4	2
5	8
6	4
7	2
8	1
9	
10	
11	
12	
13	
...	

So give
chi

Question: So now hov
of the parent of

And, due to a property of C++ integer division...

this formula works equally well for both left and right children!

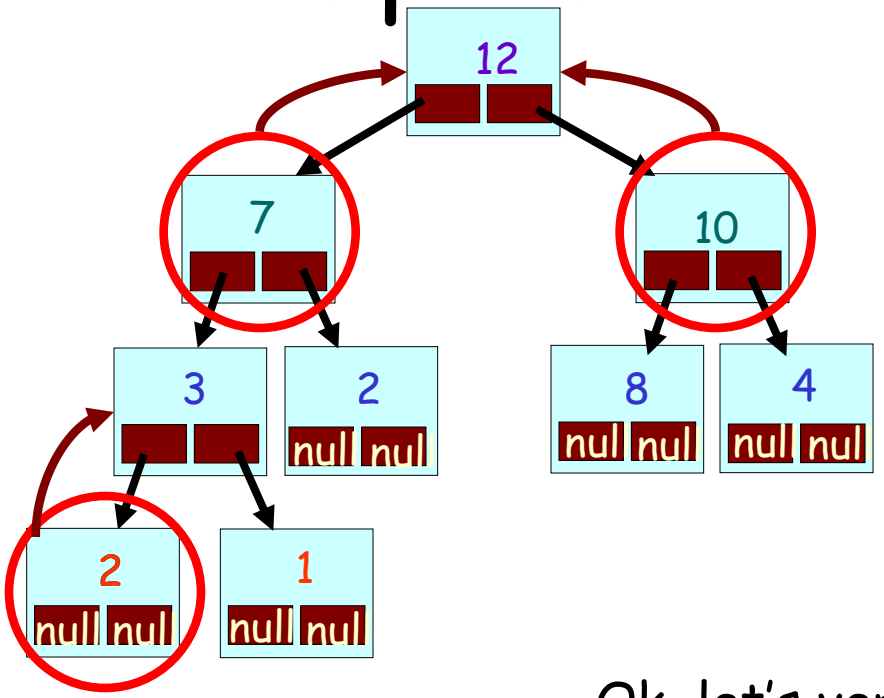
Answer: Use sy algebra!

$$\text{parent} = \frac{\text{child}-1}{2}$$

Hint: It's of ... hild(parent) = 2 * parent + 1

rightChild(parent) = 2 * parent + 2

Implementing A Heap



int count;

9

int heap[1000];

0	12
1	7
2	10
3	3
4	2
5	8
6	4
7	2
8	1
9	
10	
11	
12	
13	
...	

Ok, let's verify that it works...

The parent of slot #1 is... (1-1)/2 = 0

The parent of slot #2 is... (2-1)/2 = 0

The parent of slot #7 is... (7-1)/2 = 3

$$\text{parent} = \frac{\text{child}-1}{2}$$

Cool stool! So now we know how to locate the children of a node, find the parent of a node, and add and remove nodes!

Heap in an Array Summary

So, now we know how to store a heap in an array!

Here's a recap of what we just learned :

1. The root of the heap goes in array[0]
2. If the data for a node appears in array[i], its children, if they exist, are in these locations:
   Left child: array[2i+1]
   Right child: array[2i+2]
3. If the data for a non-root node is in array[i], then its parent is always at array[(i-1)/2] (Use integer division)

A Heap Helper Clas

```
class HeapHelper
{
   HeapHelper()              { num = 0;  }
   int GetRootIndex()        { return(0); }
   int LeftChildLoc(int i)   { return(2*i+1); }
   int RightChildLoc(int i)  { return(2*i+2); }
   int ParentLoc(int i)      { return((i-1)/2); }
   int PrintVal(int i)       { cout << a[i]; }
   void AddNode(int v)       { a[num] = v; ++num;}
private:
  int a[MAX_ITEMS];
  int num;
};
```

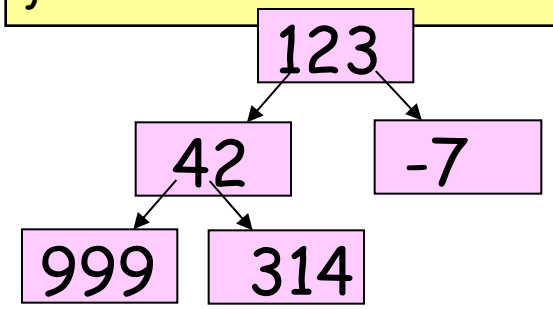
```
main()
{
   HeapH
   a.Add
   a.AddNo
   a.AddNode(-7);
   a.AddNode(999);
   a.AddNode(314);

   int i = GetRootIndex();
   PrintVal(i);
   i = LeftChildLoc(i);
   PrintVal(i);
   i = RightChildLoc(i);
   PrintVal(i);
}
```

```
-7
void AddNode(int v)
{  a[2] = -7
   a[num] = v;
   ++num;
}
```

Output: num 5

a[0]	123
[1]	42
[2]	-7
[3]	999
[4]	314



A Heap Helper Class

```
class HeapHelper
{
   HeapHelper()              { num = 0;  }
   int GetRootIndex()        { return(0); }
   int LeftChildLoc(int i)   { return(2*i+1); }
   int RightChildLoc(int i)  { return(2*i+2); }
   int ParentLoc(int i)      { return((i-1)/2); }
   int PrintVal(int i)       { cout << a[i]; }
   void AddNode(int v)       { a[num] = v; ++num;}
private:
  int a[MAX_ITEMS];
  int num;
};
```

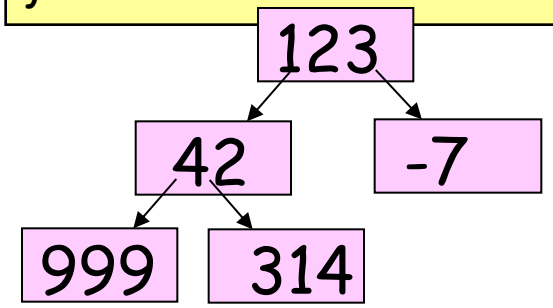
```
main()
{
   HeapHelper a;
   a.
   a
   a
   a
   a
   int
   Pri
   i =
   Pri
   i = RightChildLoc(i);
   PrintVal(i);
}
```

```
int PrintVal(int i)
{
   cout << a[i];
}
```

a[4]

a[0]	123
[1]	42
[2]	-7
[3]	999
[4]	314

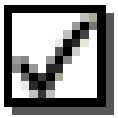
Output: num 5

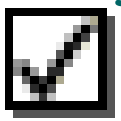


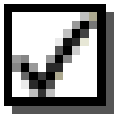
Using an Array to Implement a Heap

Ok, so now let's see how to use a simple array to implement a maxheap.

To do so, we'll need to be able to easily do the following operations:

Locate the root node of the tree… 

Locate (and delete) the bottom-most, right-most node in the tree… 

Add a new node in the bottom-most, left-most empty position in the tree… 

Easily locate the parent and children of any node in the tree… 

Extracting from a Maxheap – The Array Version!

1. If the count == 0 (it's an empty tree), return error.
2. Otherwise, heap[0] holds the biggest value. Remember it for later.
3. If the count == 1 (that was the only node) then set count=0 and return the saved value.
4. Copy the value from the right-most, bottom-most node to the root node: heap[0] = heap[count-1]
5. Delete the right-most node in the bottom-most row: count = count - 1
6. Repeatedly swap the just-moved value with the larger of its two children:
   Starting with i=0, compare and swap:
   heap[i] with heap[2*i+1] and heap[2*i+2]
7. Return the saved value to the user.

int count;

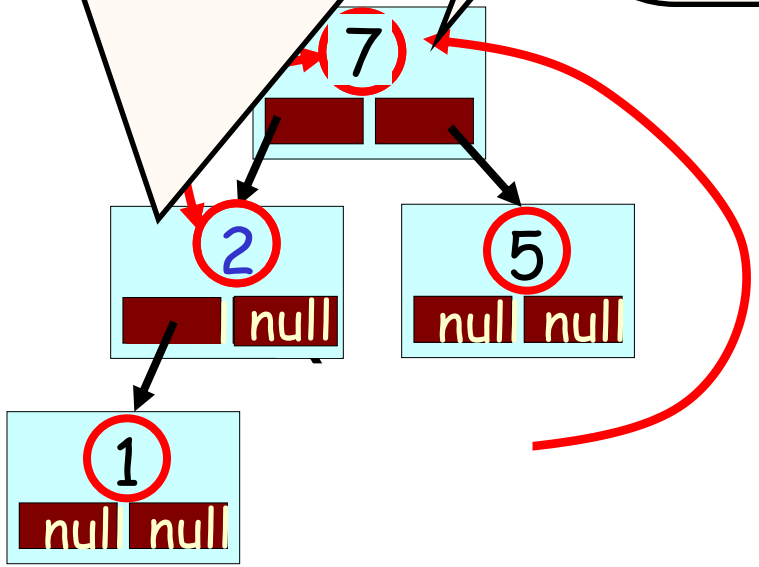
9

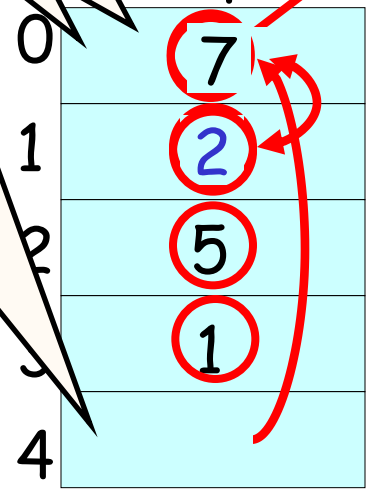
int heap[1000];

0	12
1	7
2	10
3	3
4	2
5	8
6	4
7	2
8	1
9	
10	
11	
12	
13	
…	

Finally, our heap has been "reheapified" and we're done!

...and we repeat the comparing and swapping process until our copied item is in the right place.





9

...

Adding a Node to a Maxheap – The Array Version

1. Insert a new node in the bottom-most, left-most open slot:
   heap[count] = value
   count = count + 1;
2. Compare the new value heap[i] with its parent's value: heap[(i-1)/2]
3. If the new value is greater than its parent's value, then swap them.
4. Repeat steps 2-3 until the new value rises to its proper place or we reach the top of the array.

int count;

4

int heap[10];

0	10
1	7
2	8
3	3
4	

...

Now, repeatedly compare our new

Now let's

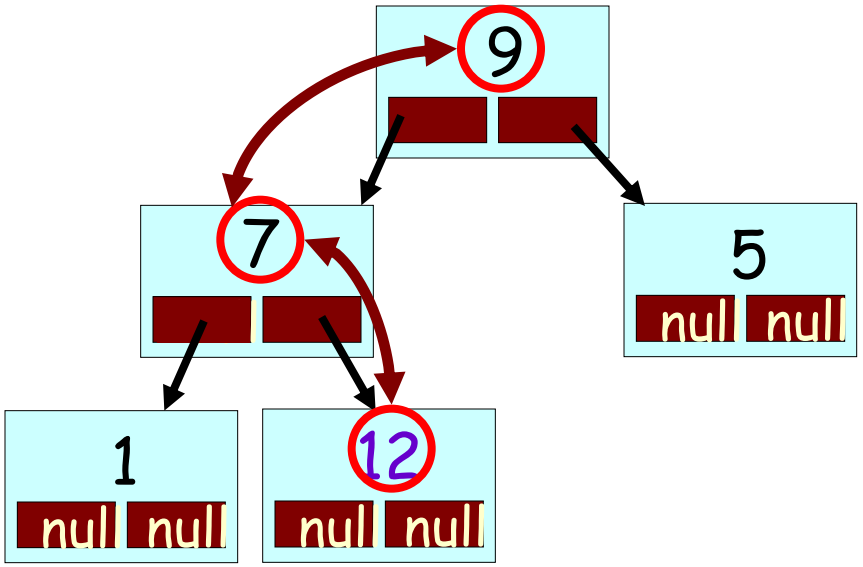
insertion

arra

Let's ad

Finally, our new value has reached its proper position in the heap and we're done…

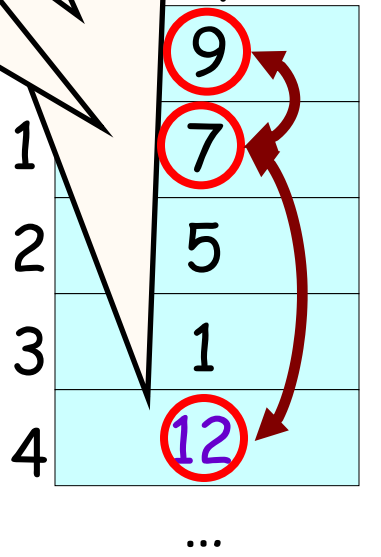
then swap



int count;

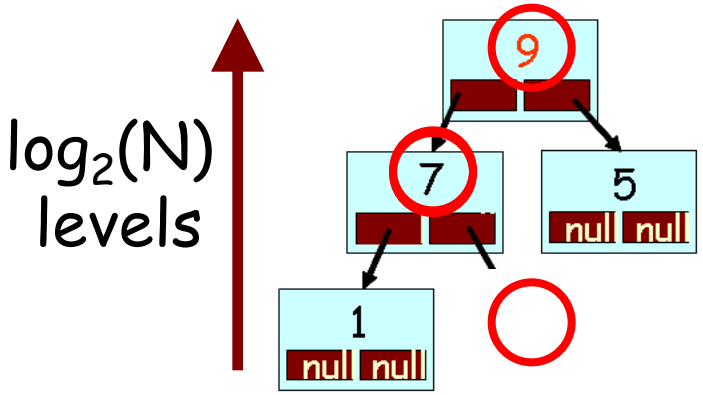
5

eap[10];



Complexity of the Heap

Question: What is the big-oh cost of inserting a new item into a heap?



Every time we insert an item, we need to keep comparing it with its parent until it reaches the right spot (This is true whether or not our heap is stored in an array!)

So in the worst case, we'll have to do $\log_2(N)$ comparisons and $\log_2(N)$ swaps.

Since our tree is a COMPLETE binary tree, it has N entries, it's guaranteed to be exactly $\log_2(N)$ levels deep.

Question: What is the big-oh cost of extracting the maximum/minimum item from a heap?

Just as with heap insertion, when we extract a value we need to bubble an item from the root down the tree.

Since the maximum number of levels in our tree is $\log_2(N)$, the worst case that this requires $\log_2(N)$ swaps.

So inserting and extracting from a heap is $O(\log_2(n))$

The Heapsort

What's the big picture?

Heapsort is $O(n*\log_2 n)$ sort that uses a maxheap in a super clever way to sort a bunch of values*.

In a nutshell, you start by taking the array that you want to sort and converting it into a maxheap.

Then you repeatedly remove the largest item from the maxheap and store it back into the array.

The first item removed from the maxheap goes into the last slot of the array, the next item removed goes into the second-to-last slot, etc. The final item (the smallest one) goes in slot 0.

Once you've removed all N items from the heap, and put them back into the array (from back → front, biggest → smallest), it's completely sorted!

Uses:

Job interviews. The sad reality is that I've never seen the heapsort used in real life. But boy is it cool.

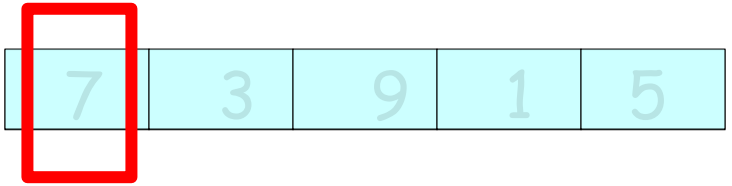
The Heapsort

Question:
How can we use a heap to sort a bunch of items?

Answer:
Here's a "naïve" way to do it…

Given an array of N numbers that we want to sort:

1. Insert all N numbers into a new maxheap
2. While there are numbers left in the heap:
   A. Remove the biggest value from the heap
   B. Place it in the last open slot of the array



The Naïve Heapsort

Question:
What's the complexity of our simple version of heapsort?

Hints:
What is the cost of inserting an item into a maxheap?
What is the cost of extracting an item from a maxheap?
How many items must be inserted and then extracted?

Insertions: N items * $\log_2(N)$ steps per item → N $\log_2(N)$ steps

Extractions: N items * $\log_2(N)$ steps per item → N $\log_2(N)$ steps

That comes to 2 * N $\log_2(N)$ total steps, or $O(N \log_2(N))$.

Not bad! But in fact there's an even faster way to use a heap to sort an array… Let's see it!

The Efficient Heapsort

In our naïve algorithm, we took every item from the array and inserted it into a separate, brand-new maxheap.

So first we built a separate maxheap from scratch, copying every one of our items over!

And then we had to remove each one from our new heap and stick them back into our original array. So SLOW!

Question:
Could we have avoided creating a whole new maxheap and moving our numbers back and forth?

Answer:
Yes! That's the way the "official" Heapsort works! Let's see!

The Efficient (Official) Heapsort

Let's update our original inefficient algorithm to turn it into the efficient (official) version.

Given an array of N numbers that we want to sort:

1. Convert our input array into a maxheap
2. While there are numbers left in the heap:
   A. Remove the biggest value from the heap
   B. Place it in the last open slot of the array

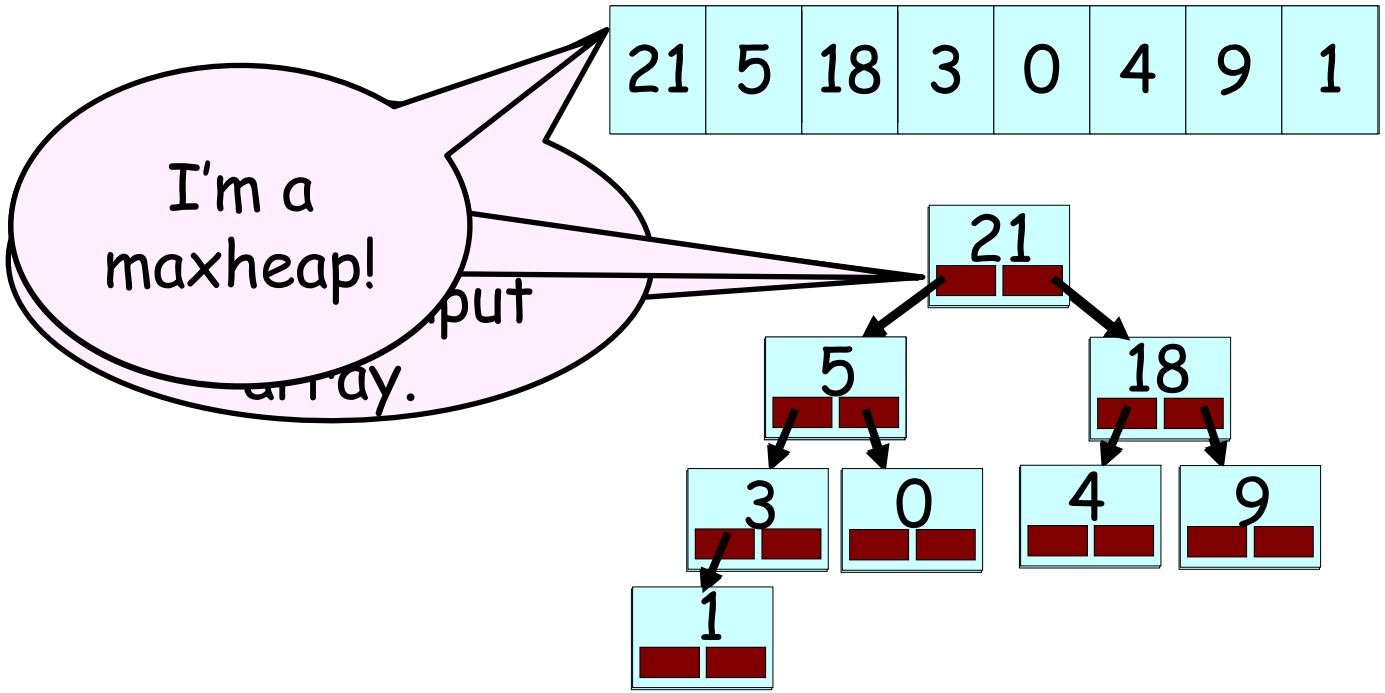
Wow! It's that simple? Yup!

We're just going to just shuffle the values around in our input array so they become a maxheap.

Then we'll use that maxheap as if it were a separate array like before. Only now everything's in just one array.

Step #1:

Convert your randomly-arranged input array into a maxheap by cleverly shuffling around the values in the array.



Let's learn the algorithm to perform this shuffling.

Step #1: Co ... y into a MaxHeap

Let's ... ray as a tree.

By last node, we mean the bottom-most, right-most node in the tree.

This corresponds to the last element in the array.

curNode

9	1	5	4	18	21

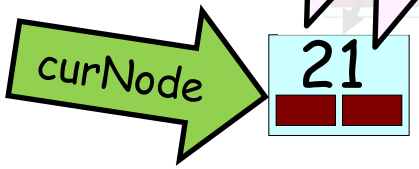
Ok, now here's th...

e: While I'm showing you the shuffling algorithm visually on a tree, it actually operates on the input array

- for (curNode = last...
  - Focus on the subt...
  - Think of this subtree as a maxheap.
  - Keep shifting the top value down until your subtree becomes a valid maxheap.

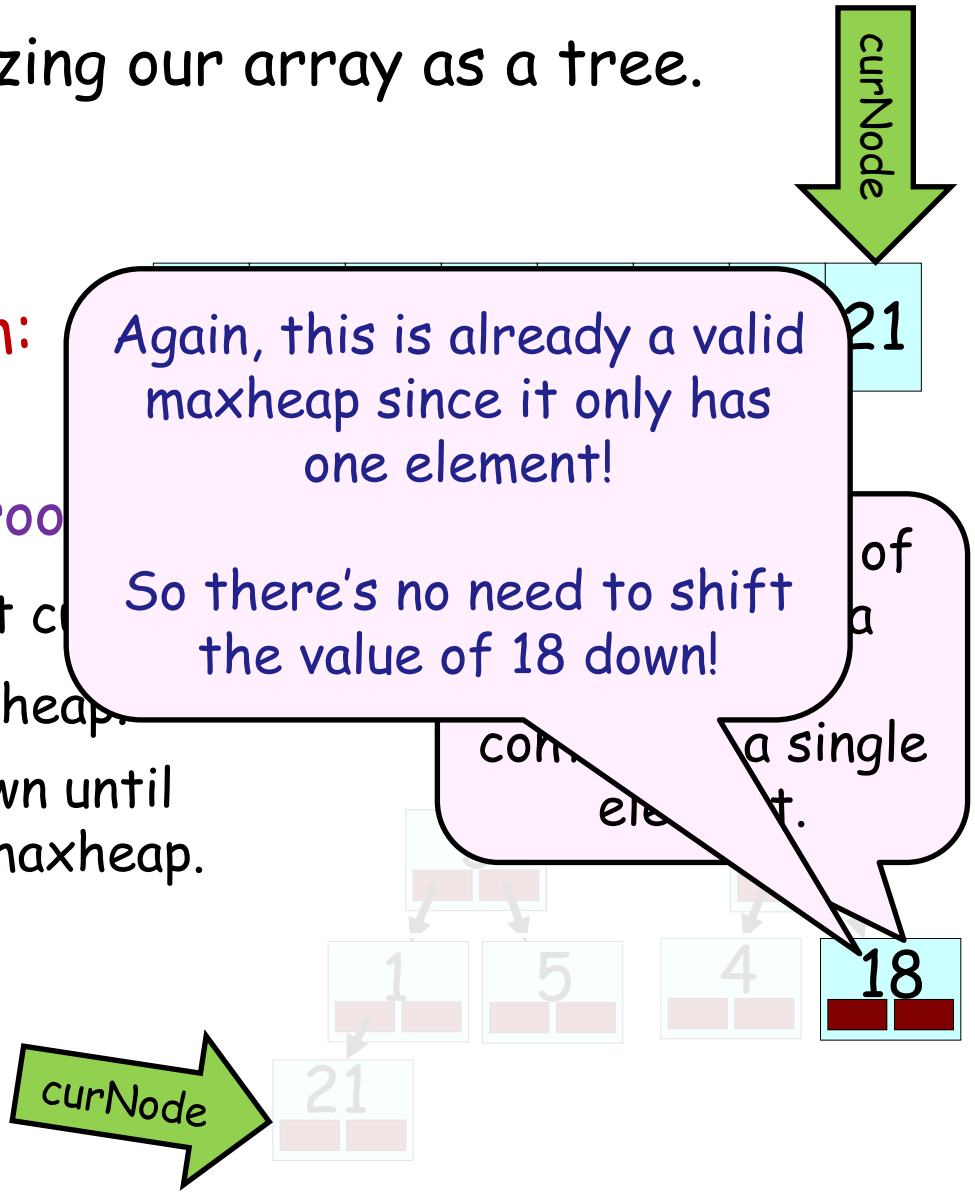
Well, this is already a valid maxheap since it only has one element!

So there's no need to shift the value of 21 down!



Step #1: Convert Your Input Array into a MaxHeap

Let's start by visualizing our array as a tree.

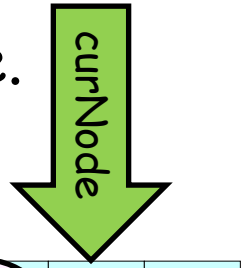


Ok, now here's the algorithm:

- for (curNode = lastNode thru roo
  - Focus on the subtree rooted at c
  - Think of this subtree as a maxheap
  - Keep shifting the top value down until your subtree becomes a valid maxheap.

Step #1: Convert Your Input Array into a MaxHeap

Let's start by visualizing our array as a tree.

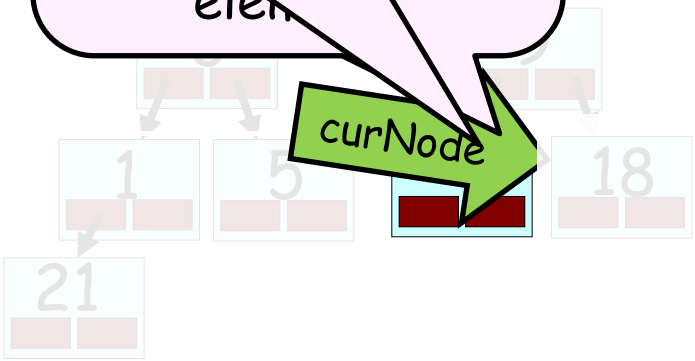


Ok, now here's the algorith

- for (curNode = lastNode thru
  - Focus on the subtree rooted
  - Think of this subtree as a max
  - Keep shifting the top value down until your subtree becomes a valid maxheap.

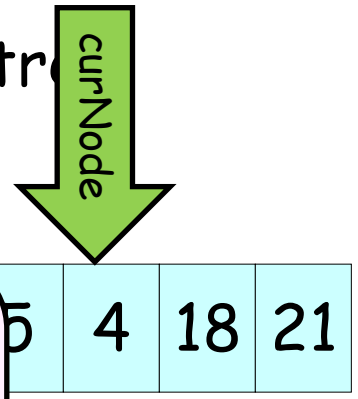
Again, this is already a valid maxheap since it only has one element!

So there's no need to shift the value of 4 down!



Step #1: Convert Your Input Array into a MaxHeap

Let's start by visualizing our array as a tr



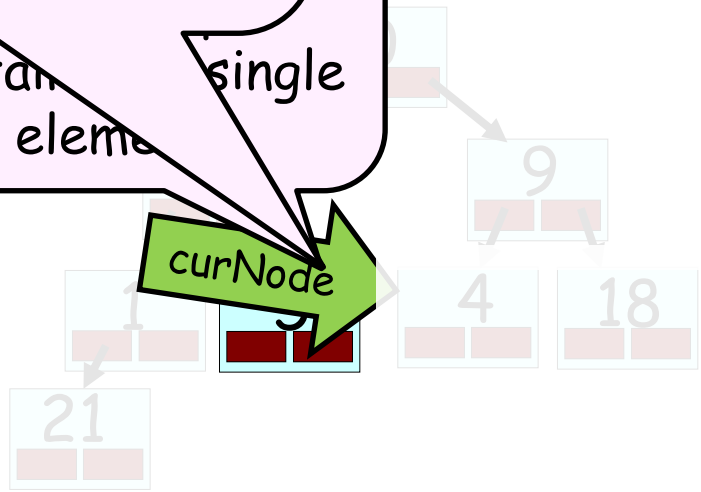
Ok, now here's the al

- for (curNode = lastNod
  - Focus on the subtree
  - Think of this subtree as a maxheap
  - Keep shifting the top value down un
    your subtree becomes a valid maxheap.

Again, this is already a valid maxheap since it only has one element!

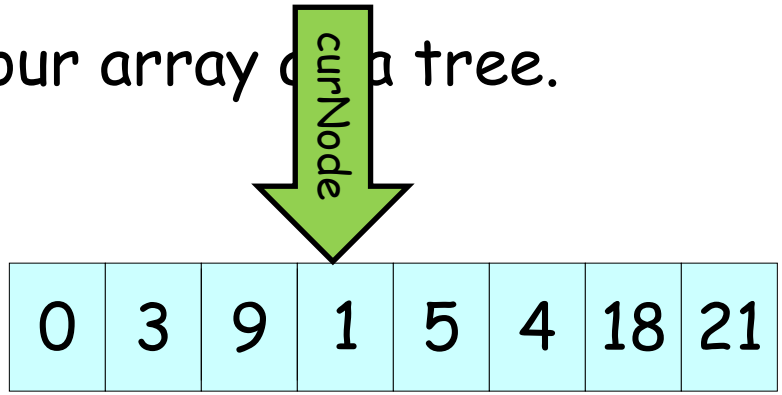
So there's no need to shift the value of 5 down!

contai single
eleme



Step #1: Convert Your Input Array into a MaxHeap

Let's start by visualizing our array as a tree.

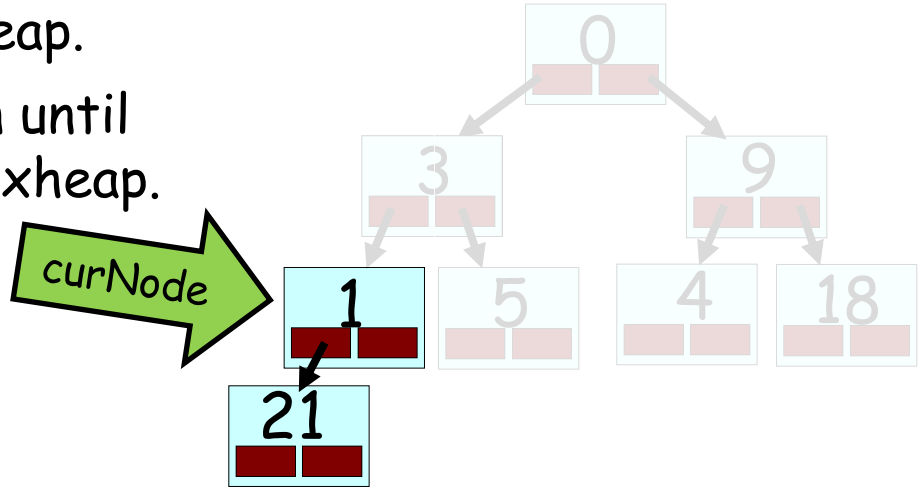


Ok, now here's the algorithm:

- for (curNode = lastNode thru rootNode):
  - Focus on the subtree rooted at curNode.

    Think of this subtree as a maxheap.

    Keep shifting the top value down until your subtree becomes a valid maxheap.



Step

o a MaxHeap

tree.

Of course, our actual algorithm would do these swaps in the array...

Now we simply use our normal heapification algorithm to turn this subtree into a maxheap.

We'll keep swapping our root value down with its larger child until it's bigger than both of its children (or hits a leaf)!

curNode

	1	5	4	18	21

Ok,

ree

ode!

for (

if it

value on

Fo

nat needs to be shifted down...

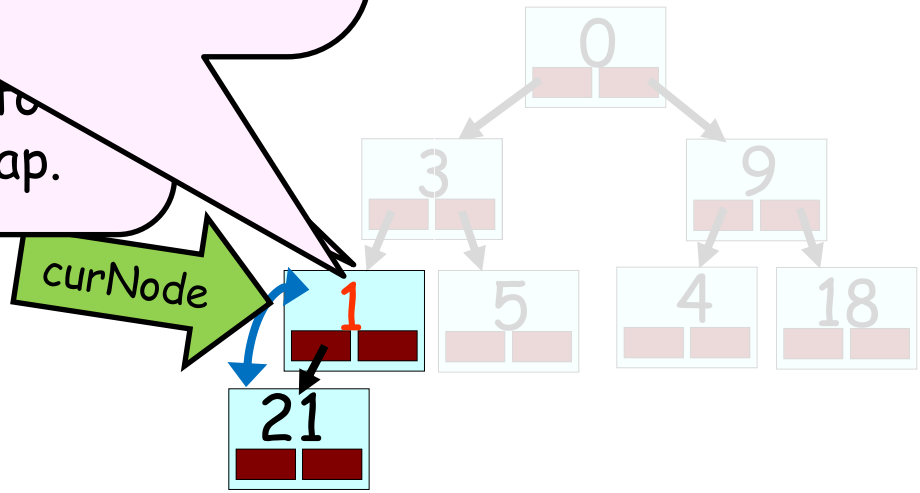
Thi

Keep shifting th

your subtree be

converted into

valid maxheap.



Step #1: Conv … a MaxHeap

Let's star … as a tree.

Of course, our actual algorithm would do these swaps in the array… ☺

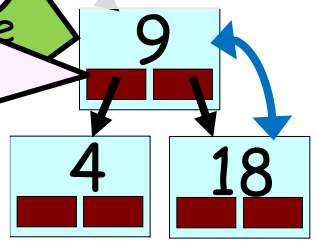
0	3	9	21	5	4	18	1

Ok, now here's the algorithm:

- for (curNode = lastNode thru root…)
  - Focus on the subtree rooted at cu…
  - Think of this subtree as a maxhea…
  - Keep shifting the top value do… your subtree becomes a valid …

Excellent! Now this subtree is a valid maxheap too.

Again, here treat this subtree as a maxheap with a new value at the top that needs to be sifted down…



Step #1: Conv … a MaxHeap

Let's star … as a tree.

Of course, our actual algorithm would do these swaps in the array… ☺

0	3	18	21	5	4	9	1

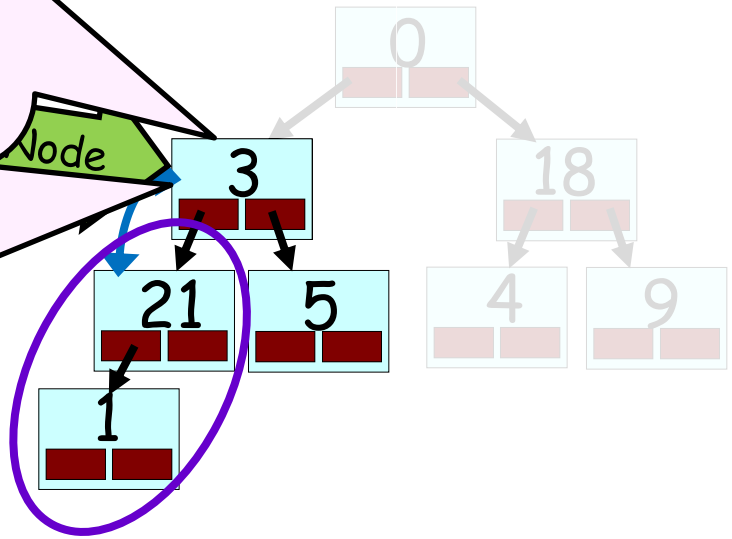
Ok, now here's the algorithm:

- for (curNo…
  - Focus on…
  - Think of…
  - Keep shi… your subtree…

Now this subtree is also a valid maxheap!

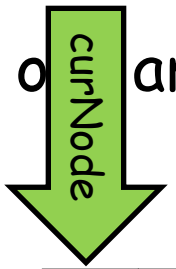
Notice how it actually built upon our earlier maxheap that we previously heapified!

…Ok, you know the drill. Let's treat this subtree as if it's a maxheap and sift the top value down appropriately.



Step #1: Convert Your Input Array into a MaxHeap

Let's start by visualizing o array as a tree.



0	21	18	3	5	4	9	1

Ok, now here's the algorithm:

for (curNode = lastNode t

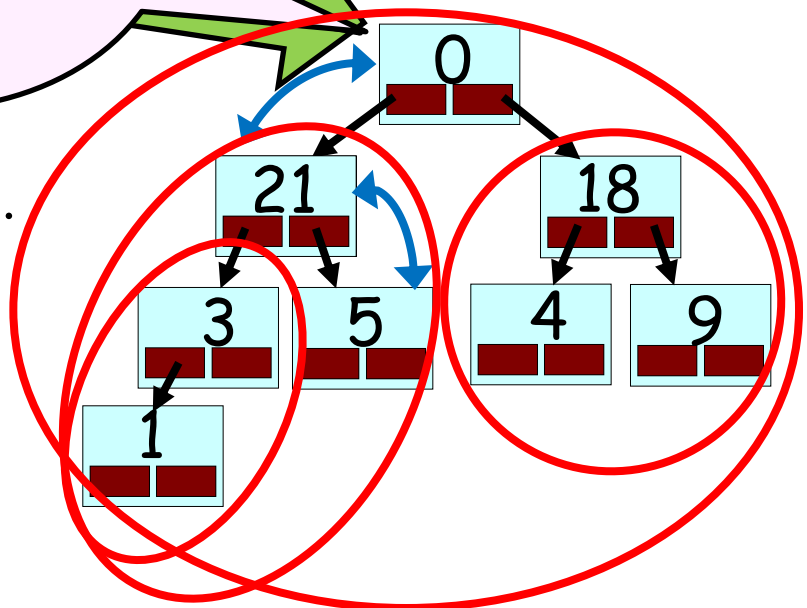
- Focus on the subtree root
- Think of this subtree as a m
- Keep shifting the top value down until your subtree becomes a valid maxheap.

I'm a valid maxheap now!

Essentially what we've done is heapify each sub-tree from the bottom-up.

As we heapify higher sub-trees, they rely upon the lower sub-trees that were heapified earlier!

Once we've finished heapifying from our root node, our entire array will hold a valid maxheap!



Eff

Alright, so we've
and our input
maxh

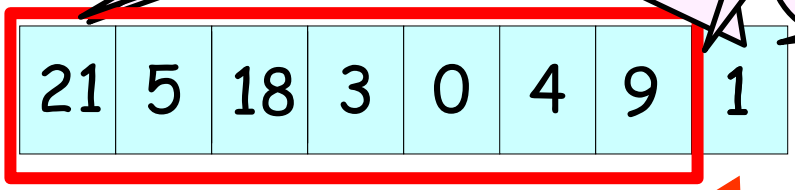
When
reheapifying
slots hold a
AND the la
array

Now the biggest
v

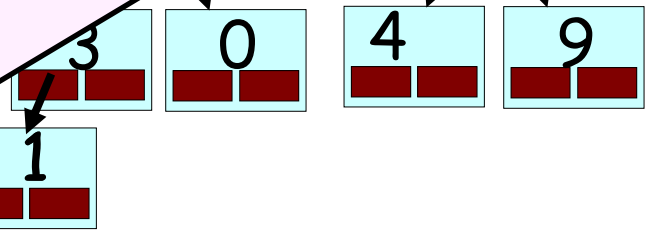
up the last
rray!

Guess what!

Step #2 of our efficient heapsort is virtually identical to Step #2 of naïve heapsort!



21



Reheapification Algorithm (same as before)

1. ~~1.~~ Copy the value from the right-most node in the bottom-most row to the root node.
2. ~~2.~~ Delete the right-most node in the bottom-most row.
3. ~~3.~~ Repeatedly swap the just-moved value with the larger of its two children until the value is greater than or equal to both of its children.

#2

axheap and re-heapify

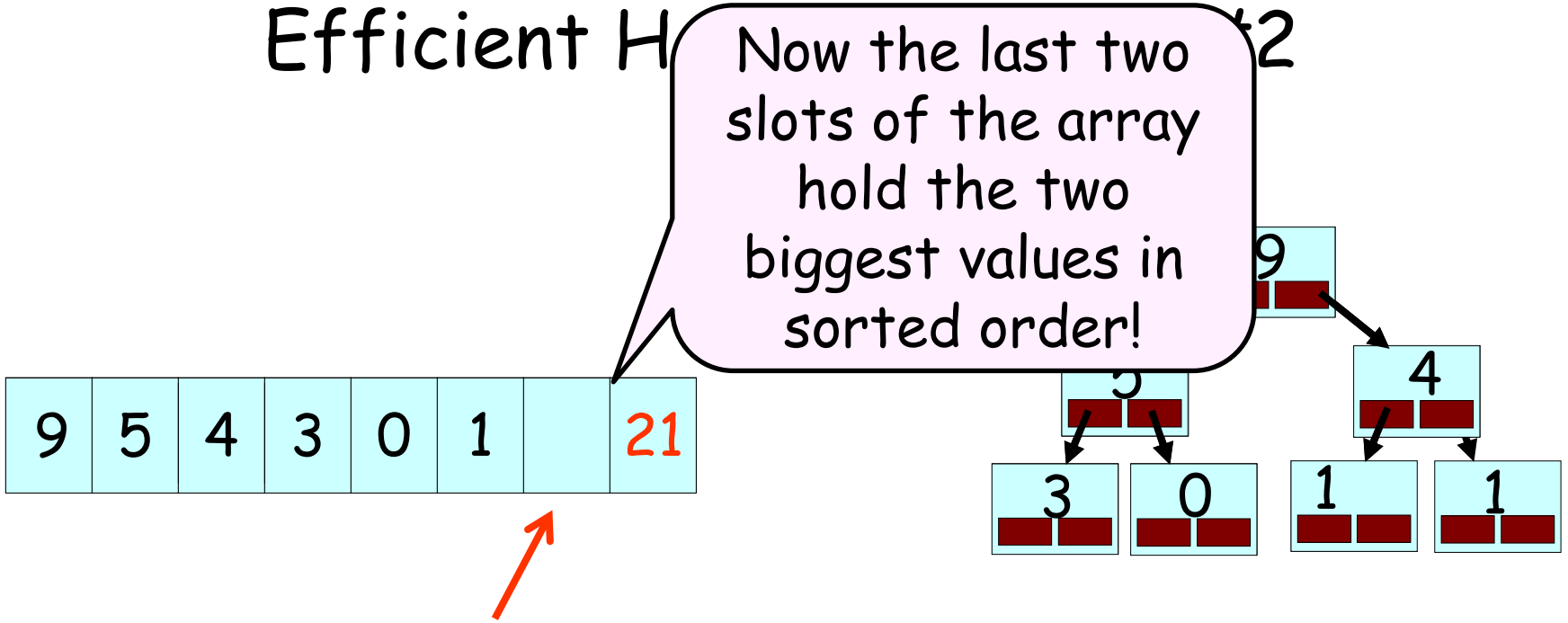
go)

y

e in it)

s freed-up slot of the array.

Efficient H…2



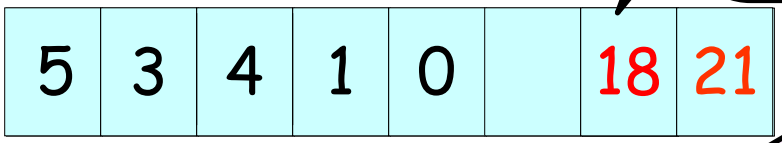
While there are numbers left in the heap:

1. Extract the biggest value from the maxheap and re-heapify (just as we learned about 20 slides ago)

   This frees up the second-to-last slot in the array (since the heap now has 1 fewer value in it)

2. Now put the extracted value into this freed-up slot of the array.

Efficient #2



9

Now the last three slots of the array hold the three biggest values, so

If you keep repeating this extraction/reheapification/insertion process the array will be completely sorted!

While there are numbers left in the heap:

1. Extract the biggest value from the maxheap and re-heapify (just as we learned about 20 slides ago)

   This frees up the third-to-last slot in the array (since the heap now has 1 fewer value in it)

2. Now put the extracted value into this freed-up slot of the array.

Big-O of Heapsort!

If you think it should be O(N log2N), I did too. ☺ Come to office hours for an explanation.

Step #1:

First we take our N-item array (shown here as a tree) and convert it into a maxheap.

We do this from the bottom up by converting successively larger subtrees into maxheaps until the entire tree has been converted.

Step #1 has a Big-O of O(N).

In other words, we can convert a random array into a maxheap in just O(N) steps!

+

Step #2:

Then we repeatedly extract the j^{th} largest item from the maxheap and place that item back into the array, j slots from the end.

Step #2 has a Big-O of $O(N \log_2 N)$.

Why? Each time we remove an item from the maxheap, it takes $\log_2 N$ steps. We perform this operation N times to sort the entire array.

Therefore, Heapsort is $O(N + N \log_2 N)$, which as you know, is just $O(N \log_2 N)$.